

# DASHBOARD

Document d'architecture et de benchmark technique

---

Adams Kamate & Mohammad Amin Hammami

Justification des choix technologiques et plan de réalisation

# Sommaire

|   |                                                                                       |
|---|---------------------------------------------------------------------------------------|
| 1 | <b>Rappel des exigences du sujet</b><br>14 contraintes extraites (C1 à C14)           |
| 2 | <b>Benchmark et justification</b><br>Front, back, base, cache, auth, API, déploiement |
| 3 | <b>Architecture applicative</b><br>Composants                                         |

1. CONTRAINTES

# Rappel des exigences du sujet (1/2)

Avant de justifier quoi que ce soit, il faut lister ce que le sujet impose réellement. Chaque choix technique est ensuite rattaché à l'une de ces contraintes

| #   | Exigence du sujet                                          | Conséquence technique                                       |
|-----|------------------------------------------------------------|-------------------------------------------------------------|
| C1  | docker-compose build et docker-compose up obligatoires     | Toute la stack doit être conteneurisable                    |
| C2  | Serveur sur le port 8080 répondant à GET /about.json       | Backend HTTP exposant un JSON à structure imposée           |
| C3  | Inscription + confirmation de compte avant accès           | Auth interne (email/mot de passe), mail de confirmation     |
| C4  | Authentification (credentials ou OIDC)                     | JWT / session                                               |
| C5  | Autorisation via OAuth 2.0 pour les APIs externes          | Implémentation manuelle du flow OAuth2 (libs interdites)    |
| C6  | Lier le compte tiers à un utilisateur de la plateforme     | Table de liaison user / service / tokens                    |
| C7  | (1 + X) services et (3 × X) widgets pour X étudiants       | Architecture extensible : pas de refonte par service ajouté |
| C8  | Chaque widget doit avoir des paramètres configurables      | Stockage de params hétérogènes par instance                 |
| C9  | Timer : chaque instance communique son refresh rate        | Système de tâches planifiées / polling par instance         |
| C10 | Ajouter / reconfigurer / déplacer / supprimer une instance | CRUD complet + persistance de la position                   |
| C11 | Deux instances du même widget, configs différentes         | La config appartient à l'instance, pas au type de widget    |
| C12 | Interface responsive, bonne UX, accessibilité              | Framework front avec écosystème UI mature                   |
| C13 | Données personnelles et credentials sensibles              | Tokens chiffrés, mots de passe hashés, pas de secret client |
| C14 | README complet + justification professionnelle             | Ce document + documentation continue                        |

# Frontend : Next.js (React) + Tailwind CSS

| Critère                         | Next.js (React)       | React seul (Vite)        | Vue.js                 | Angular                            |
|---------------------------------|-----------------------|--------------------------|------------------------|------------------------------------|
| Poids                           | Moyen                 | Léger                    | Très léger             | Lourd                              |
| Difficulté d'apprentissage      | Facile si React connu | Facile                   | Facile                 | Difficile, demande de l'expérience |
| Communauté & documentation      | Massive               | Massive                  | Bonne mais plus petite | Entreprise, libs Google            |
| Maintenabilité en équipe        | Très bonne            | Bonne, tout à structurer | Bonne                  | Très bonne si maîtrisé             |
| Routing intégré                 | Oui                   | Non (lib externe)        | Non (lib externe)      | Oui                                |
| Routes serveur (callback OAuth) | Oui                   | Non                      | Non                    | Non                                |
| Recommandé par le sujet         | Oui (React)           | Oui (React)              | Oui                    | Oui                                |

Choix : Next.js; la différence décisive est la route serveur, qui permet de traiter le callback OAuth sans exposer de secret au navigateur (C13).

Styling : Tailwind CSS, recommandé par le sujet, au lieu du Bootstrap (identité visuelle générique) et du CSS pur (trop lent pour couvrir C12).

# Frontend : pourquoi le Next.js (React) au détriment des autres

1

## Au détriment d'Angular

La courbe d'apprentissage et la lourdeur du framework sont disproportionnées par rapport à la taille du projet. Le sujet recommande explicitement des solutions simples et de ne pas sur-architecturer (« Focus on clear, minimal solutions »).

2

## Au détriment de Vue

Vue est plus léger et rapide, mais sa communauté et sa documentation sont plus réduites. Sur un projet à deadline serrée avec des problèmes d'intégration OAuth et d'APIs tierces, la quantité de ressources disponibles prime sur quelques kilo-octets de bundle.

3

## Au détriment de React seul

Next.js apporte le routing natif et surtout les routes serveur, très utiles pour gérer les callbacks OAuth sans exposer de secret côté navigateur (C13). C'est la différence décisive entre les deux.

**Principe général : à contrainte fonctionnelle égale, le critère qui départage est la sécurité du flow OAuth, puis le volume de ressources disponibles pour débloquer l'équipe.**

# Backend : Node.js (Express) vs Go vs Python

| Critère                             | Node.js (Express)                | Go                      | Python (Django/FastAPI) |
|-------------------------------------|----------------------------------|-------------------------|-------------------------|
| Vitesse d'exécution                 | Modérée                          | La plus rapide          | La plus lente           |
| Communauté                          | Massive                          | Très large et active    | Large                   |
| Temps de build                      | Rapide                           | Rapide                  | Rapide                  |
| Recommandé par le sujet             | Oui                              | Non listé               | Oui                     |
| Un seul langage front/back          | Oui (TypeScript)                 | Non                     | Non                     |
| Manipulation JSON (C2)              | Native                           | Structs + tags, verbeux | Native                  |
| Appels concurrents vers N APIs (C9) | Très bon (event loop, I/O-bound) | Excellent (goroutines)  | Bon avec async          |
| OAuth2 à la main (C5)               | fetch natif suffit               | net/http suffit         | httpx / requests        |
| Vitesse de développement            | Rapide                           | Plus verbeux            | Rapide                  |
| Conso mémoire / CPU                 | Faible à modérée                 | Très faible             | Élevée                  |

Choix : Node.js + Express, au détriment de Go et de Python.

# Backend : le facteur limitant réel du projet

Le facteur limitant n'est pas la performance : le trafic attendu est celui d'une démo de keynote, pas d'une production à grande échelle. Le facteur limitant est le volume de fonctionnalités à livrer (C7 : jusqu'à  $(1+X)$  services et  $(3 \times X)$  widgets, chacun avec son intégration d'API et son flow OAuth fait main).

### Au détriment de Go

Go gagnerait là où la performance brute et la concurrence sont critiques. Ici la charge est I/O-bound (attendre des réponses d'APIs externes), un domaine où l'event loop de Node suffit largement - l'avantage des goroutines existe mais ne produit aucun gain perceptible à cette échelle. En contrepartie : un second langage à maîtriser, une syntaxe plus verbeuse, et une absence de la liste des technos recommandées.

### Au détriment de Python

L'écosystème data/analytics (pandas, numpy) est son principal atout, or ce projet ne fait aucun calcul statistique - il agrège et affiche des données d'APIs tierces. Cet avantage ne sert à rien ici, alors que le coût (langage supplémentaire, performance plus faible) reste.

### En faveur de Node

Un seul langage sur toute la stack permet de partager les types TypeScript entre front et back (une interface Widget définie une fois), ce qui réduit mécaniquement les erreurs d'intégration. La manipulation JSON native rend la génération de about.json (C2) triviale.

# Base de données : PostgreSQL

| Critère                              | PostgreSQL       | MySQL        | MongoDB                           |
|--------------------------------------|------------------|--------------|-----------------------------------|
| Support JSON typé et indexable       | JSONB, excellent | JSON basique | Natif (document)                  |
| Relations users / services / widgets | Excellent        | Excellent    | Faible (pas de jointures natives) |
| Requêtes analytiques complexes       | Excellent        | Correct      | Limité                            |
| Conformité ACID                      | Stricte          | Bonne        | Plus faible                       |
| Extensions                           | Très riches      | Peu          | —                                 |
| Image Docker officielle (C1)         | Oui              | Oui          | Oui                               |

### Au détriment de MySQL

C8/C11 imposent des paramètres hétérogènes par instance ({city: "Paris"} vs {link, number}). JSONB stocke et interroge ces structures variables dans une colonne typée et indexable ; MySQL demanderait une table EAV lourde ou du JSON texte non interrogeable.

### Au détriment de MongoDB

Le modèle est fondamentalement relationnel : un utilisateur souscrit à N services, chaque service expose N widgets, chaque widget a N instances. PostgreSQL donne le relationnel ET le JSON ; MongoDB donnerait seulement le JSON.

Décision explicite : pas de base Time-Series (InfluxDB / TimescaleDB). Le sujet ne demande aucun historique temporel - les widgets affichent l'état courant d'une source, pas son évolution stockée. L'ajouter serait de la sur-ingénierie et un service Docker de plus à maintenir pour rien.

# Cache et file de tâches : Redis + BullMQ

| Critère                         | Redis                   | Memcached            | RabbitMQ       |
|---------------------------------|-------------------------|----------------------|----------------|
| Cache en RAM                    | Oui                     | Oui                  | Non            |
| File de tâches / jobs planifiés | Oui                     | Non                  | Oui            |
| Structures de données riches    | Oui                     | Clé-valeur seulement | —              |
| Persistance optionnelle         | Oui                     | Non                  | Oui            |
| Services Docker ajoutés         | 1 (fait les deux rôles) | 1 (cache seul)       | 1 (jobs seuls) |

## Pourquoi Redis

Le projet a deux besoins distincts : cacher les réponses d'APIs externes et planifier les rafraîchissements du Timer (C9). Redis couvre les deux avec un seul conteneur, là où Memcached ou RabbitMQ obligerait à en ajouter un second. Moins de services = un docker-compose.yml plus simple (C1).

## Le cache est une nécessité

Les APIs visées (GitHub, Steam, météo) appliquent des quotas. Sans cache, dix utilisateurs affichant le même widget déclencheraient dix appels identiques et feraient atteindre le rate-limit.

## Pourquoi BullMQ

Celery impose un runtime Python, incohérent avec un backend Node. node-cron ne gère ni la persistance des jobs (tout est perdu au redémarrage du conteneur) ni la reprise sur erreur. BullMQ s'appuie sur le Redis déjà présent et gère les jobs répétables par instance.

# Authentification et autorisation

| Mécanisme                 | Rôle dans le sujet                                | Choix technique                                       |
|---------------------------|---------------------------------------------------|-------------------------------------------------------|
| Authentification (C3, C4) | Identifier l'utilisateur de la plateforme         | Email + mot de passe hashé (bcrypt/argon2) + JWT      |
| Autorisation (C5, C6)     | Permettre aux widgets d'accéder aux APIs externes | OAuth 2.0 Authorization Code, implémenté manuellement |

**Contrainte majeure : aucune librairie ne doit implémenter OAuth à votre place - Passport.js, NextAuth, simple-oauth2 sont exclus. Seul un client HTTP générique (fetch natif) est utilisé.**

## Le flow à implémenter à la main

- 1 Générer un paramètre state aléatoire (anti-CSRF), stocké côté serveur ou en session Redis
- 2 Rediriger vers l'URL d'autorisation du provider (client\_id, redirect\_uri, scope, state)
- 3 Recevoir le callback et vérifier que le state correspond à celui émis
- 4 POST serveur-à-serveur vers le token endpoint : échanger le code contre un access\_token
- 5 Stocker le token chiffré en base, lié à l'utilisateur de la plateforme (C6)
- 6 Rafraîchir le token à expiration via le refresh\_token

### Points de sécurité (C13)

- Le client\_secret ne transite jamais par le navigateur : l'échange du code se fait exclusivement côté serveur.
- Les tokens en base sont chiffrés (AES-GCM avec une clé d'application), jamais en clair.
- JWT au détriment des sessions serveur : stateless, pas de store à ajouter — stocké en cookie httpOnly pour résister au XSS.

# API REST et déploiement Docker Compose

Choix : REST, au détriment de GraphQL et des WebSockets

Déploiement : Docker Compose (imposé, C1 et C2)

| Service    | Image / build      | Port | Rôle                        |
|------------|--------------------|------|-----------------------------|
| server     | build backend      | 8080 | API REST, OAuth, about.json |
| client_web | build frontend     | 8081 | Interface Next.js           |
| db         | postgres:16-alpine | 5432 | Données persistantes        |
| redis      | redis:7-alpine     | 6379 | Cache + file de jobs        |
| worker     | build backend      | —    | Exécution des jobs du Timer |

Au détriment de GraphQL

Le sujet impose déjà un endpoint REST à structure fixe (GET /about.json, C2). Introduire GraphQL à côté signifierait maintenir deux paradigmes d'API en parallèle, pour un bénéfice nul à cette échelle.

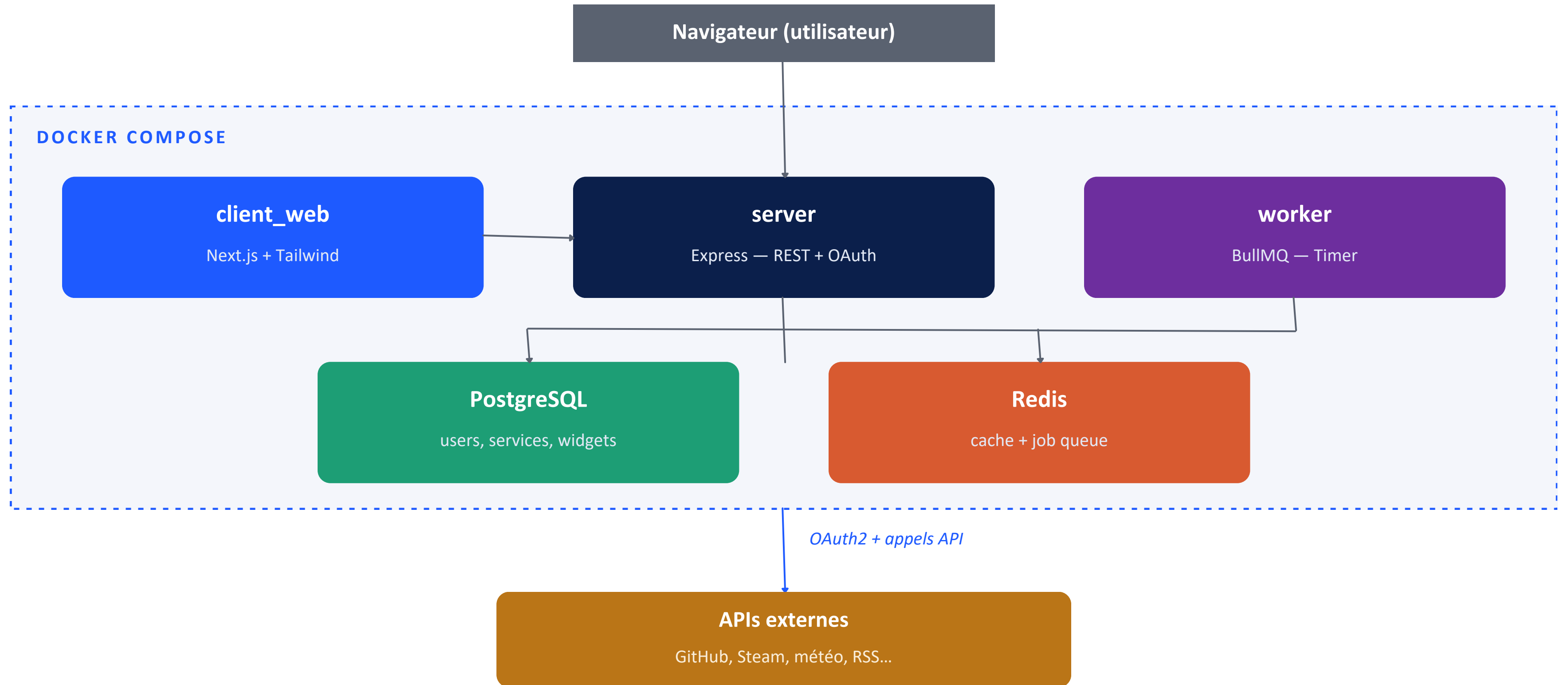
Au détriment des WebSockets

Le sujet décrit explicitement un mécanisme de polling par Timer. Les WebSockets répondraient à un besoin de push temps réel non demandé — à garder comme bonus, pas comme base.

## Points d'attention

- Volumes nommés pour PostgreSQL, sinon les données disparaissent à chaque docker-compose down.
- depends\_on avec healthchecks : le backend doit attendre que PostgreSQL soit réellement prêt, pas seulement démarré.
- Tous les secrets (client\_id/secret OAuth, clé JWT) en variables d'environnement, jamais en dur ni committés, fournir un .env.example documenté.

# Vue d'ensemble des composants



# Modèle de données

Six tables suffisent à couvrir l'ensemble des contraintes fonctionnelles.

| Table            | Colonnes principales                                                                   | Contrainte  |
|------------------|----------------------------------------------------------------------------------------|-------------|
| users            | id, email, password_hash, is_verified, verification_token, role, created_at            | C3, C4      |
| services         | id, name, requires_auth (bool), oauth_config                                           | C7          |
| user_services    | user_id, service_id, access_token (chiffré), refresh_token, expires_at                 | C5, C6, C13 |
| widget_types     | id, service_id, name, description, params_schema (JSONB)                               | C2, C8      |
| widget_instances | id, user_id, widget_type_id, params (JSONB), refresh_rate, position_x/y, width, height | C8–C11      |
| widget_cache     | instance_id, data (JSONB), fetched_at                                                  | C9          |

## Le point d'architecture le plus important : la séparation widget\_types / widget\_instances

- widget\_types décrit ce qu'un widget sait faire (nom, paramètres possibles), c'est ce qui alimente directement about.json (C2).
- widget\_instances décrit une occurrence configurée sur le dashboard d'un utilisateur, avec ses propres params.
- C'est exactement ce qui satisfait C11 : deux instances de city\_temperature, l'une {city: "Paris"}, l'autre {city: "Tokyo"}, affichent des données différentes en même temps.

# Récapitulatif des choix

| Couche          | Choix retenu           | Écarté au profit de ce choix           | Raison décisive                                                   |
|-----------------|------------------------|----------------------------------------|-------------------------------------------------------------------|
| Frontend        | Next.js + Tailwind     | React seul, Vue, Angular               | Routes serveur pour les callbacks OAuth, communauté maximale      |
| Backend         | Node.js + Express      | Go, Python                             | Langage unique front/back, vitesse de dev sur un projet I/O-bound |
| Base de données | PostgreSQL             | MySQL, MongoDB                         | JSONB pour les params hétérogènes + modèle relationnel            |
| Time-series     | Aucune                 | InfluxDB, TimescaleDB                  | Aucun historique temporel demandé par le sujet                    |
| Cache + jobs    | Redis + BullMQ         | Memcached, RabbitMQ, Celery, node-cron | Un seul conteneur pour deux besoins, jobs persistants             |
| API             | REST                   | GraphQL, WebSockets                    | Format déjà imposé par le sujet, polling suffisant                |
| Auth            | JWT + OAuth2 fait main | Passport.js, NextAuth (interdits)      | Contrainte explicite du projet                                    |
| Déploiement     | Docker Compose         | —                                      | Imposé par le sujet                                               |

Chaque ligne est justifiée dans le document complet : ce qui a été écarté, et sur quel critère précis du sujet la décision repose.